IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

MAVANI VIRAJBHAI VINUBHAI
4-11-2025

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies

Data Sourcing & Cleaning: We collected data using **SpaceX REST API** and **Wikipedia Web Scraping**. The data was cleaned, filtered for 'Falcon 9', and a binary target variable (Class: 1=Success, 0=Failure) was created.

Analysis Tools: We performed **Exploratory Data Analysis (EDA)** using **SQL** queries and visualizations

Predictive Modeling: The dataset was prepared using **One-Hot Encoding** and scaling. Four classification models (LR, SVM, DT, k-NN) were trained and optimized using **GridSearchCV**.

Summary of all results

Key Findings (EDA): The success rate has **dramatically improved over time** (especially after 2013-2015). **Orbit Type** (e.g., GTO has the lowest success rate) and **Launch Site** (KSC LC 39A has the highest success ratio) are the strongest predictors.

Interactive Insights: The interactive map clearly showed the success/failure ratio for each site, while the dashboard provided dynamic filtering by Payload Mass and Launch Site.

Model Performance: All four predictive models achieved an **identical high test accuracy of 83.33%** in predicting whether the first stage would land successfully or not, demonstrating a high ability to classify landing outcomes.

Introduction

- Project background and context

"SpaceX's Falcon 9 launches cost over $60 million per flight. Successfully landing and reusing the rocket's first stage is a key part of their strategy to dramatically reduce the cost of space access. This reusability is critical for making space exploration more affordable."

- Problems you want to find answers

The primary goal of this project is to build a machine learning model that can predict whether the Falcon 9 first stage will land successfully."
"We will also analyze the data to determine which factors (like Launch Site, Orbit, and Payload Mass) are the most influential in predicting a successful landing."

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:

We collected data using the SpaceX REST API, calling additional endpoints with helper functions to extract nested rocket, payload, and core details from their IDs. The resulting dataset was then cleaned by filtering for 'Falcon 9' launches and imputing the mean for missing Payload Mass values.

- Perform data wrangling
- We performed data wrangling by converting the categorical Outcome column into a binary Class target variable (1 for success, 0 for failure). This Class column, which has a 66.7% success rate, is now the target for our prediction models.
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models

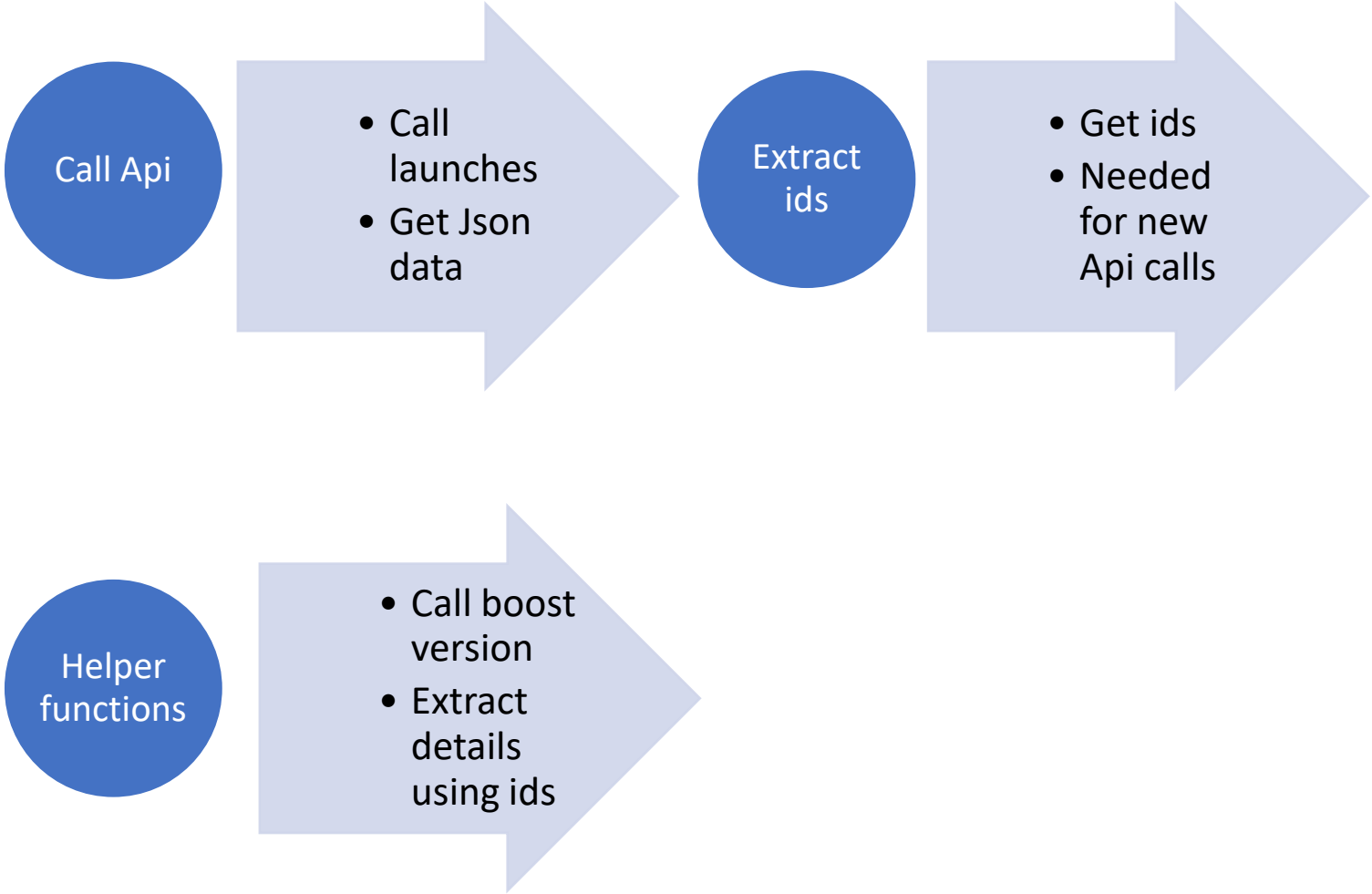
Data Collection

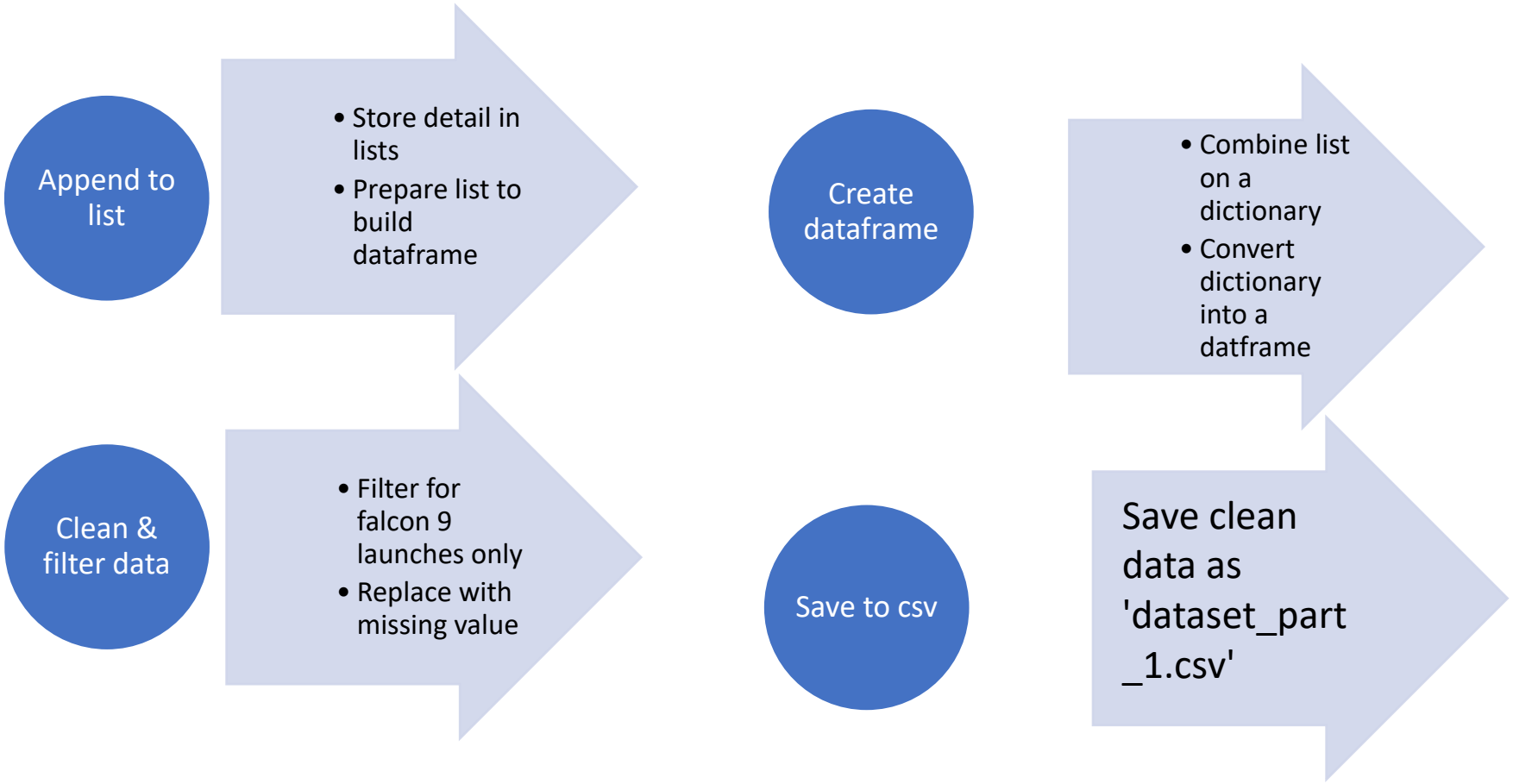
- Describe how data sets were collected.

1. **Primary Source:** Data was collected from the public **SpaceX REST API** using Python's requests library
2. **Initial fetch**: we first made a get request to the v4/launchsite/past endpoint to get a Json list of all past launches
3. **Data Extraction (The Challenge):** The initial JSON file contained IDs for key details (like rocket, payload, core) but not the actual data (like the rocket's name or the core's landing status).
4. **Helper Functions:** We created several Python "helper functions" (like getBoosterVersion, getPayloadData, getCoreData) to solve this.
5. **Iterative API Calls:** These functions looped through every launch, took its ID, and made *new* API calls to other endpoints (e.g., v4/rockets/, v4/payloads/) to get the specific details.

6. Final DataFrame: All the extracted data (like BoosterVersion, PayloadMass, Orbit, Outcome) was stored in Python lists and then combined into a single, clean **Pandas DataFrame** for analysis.

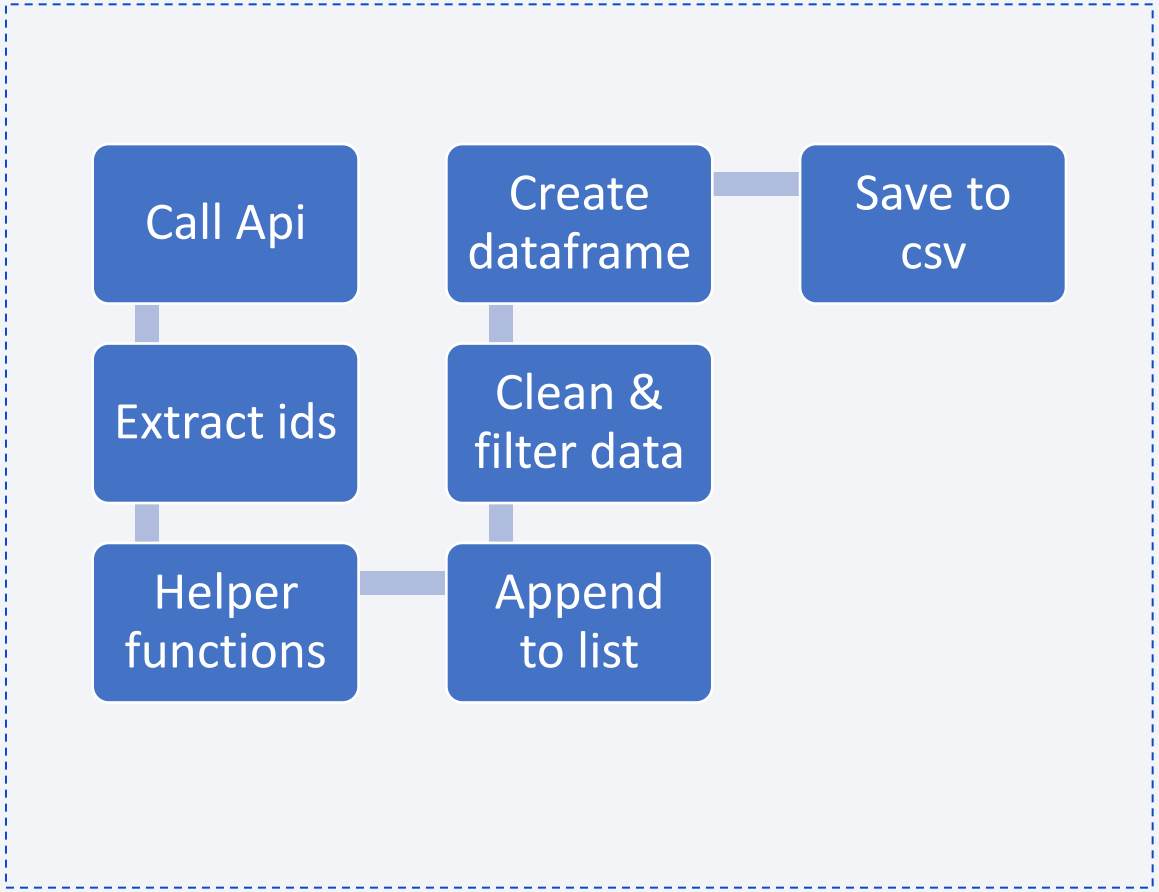
You need to present your data collection process use key phrases and flowcharts




Append to list
• Store detail in lists
• Prepare list to build dataframe
Create dataframe
• Combine list on a dictionary
• Convert dictionary into a datframe
Clean & filter data
• Filter for falcon 9 launches only
• Replace with missing value
Save to csv
Save clean data as 'dataset_part _1.csv'

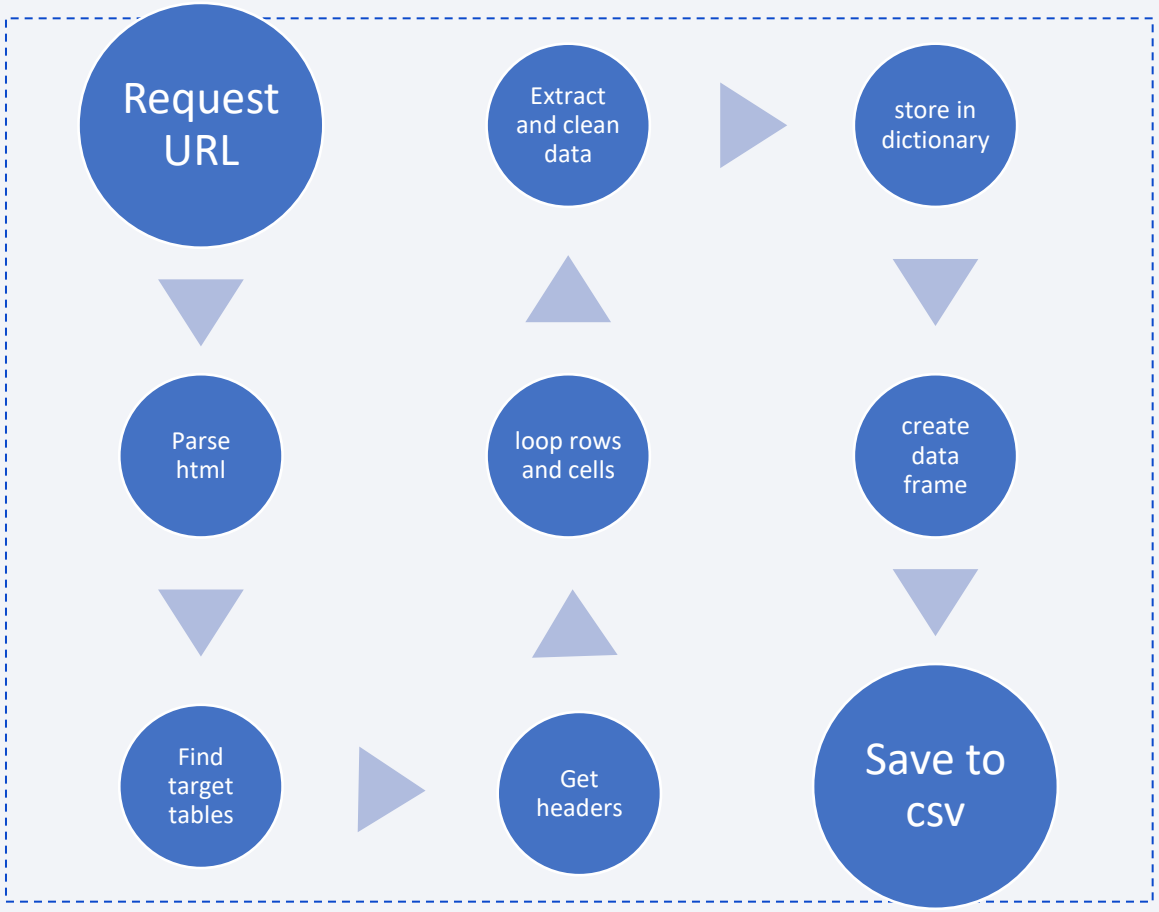
Data Collection – SpaceX API

- Api data fetch
- Nested data extraction
- Data aggregation
- Data cleaning
- https://github.com/viraj01-coder/data-science-capstone/blob/main/jupyter-labs-spacex-data-collection-api.ipynb



Data Collection - Scraping

- Request URL
- Parse HTML
- Locate Table
- Extract Headers
- Iterate & Extract
- Clean & Store
- Create Data Fram
- https://github.com/viraj01-coder/data-science-capstone/blob/main/jupyter-labs-webscraping%20(2)%20(1).ipynb

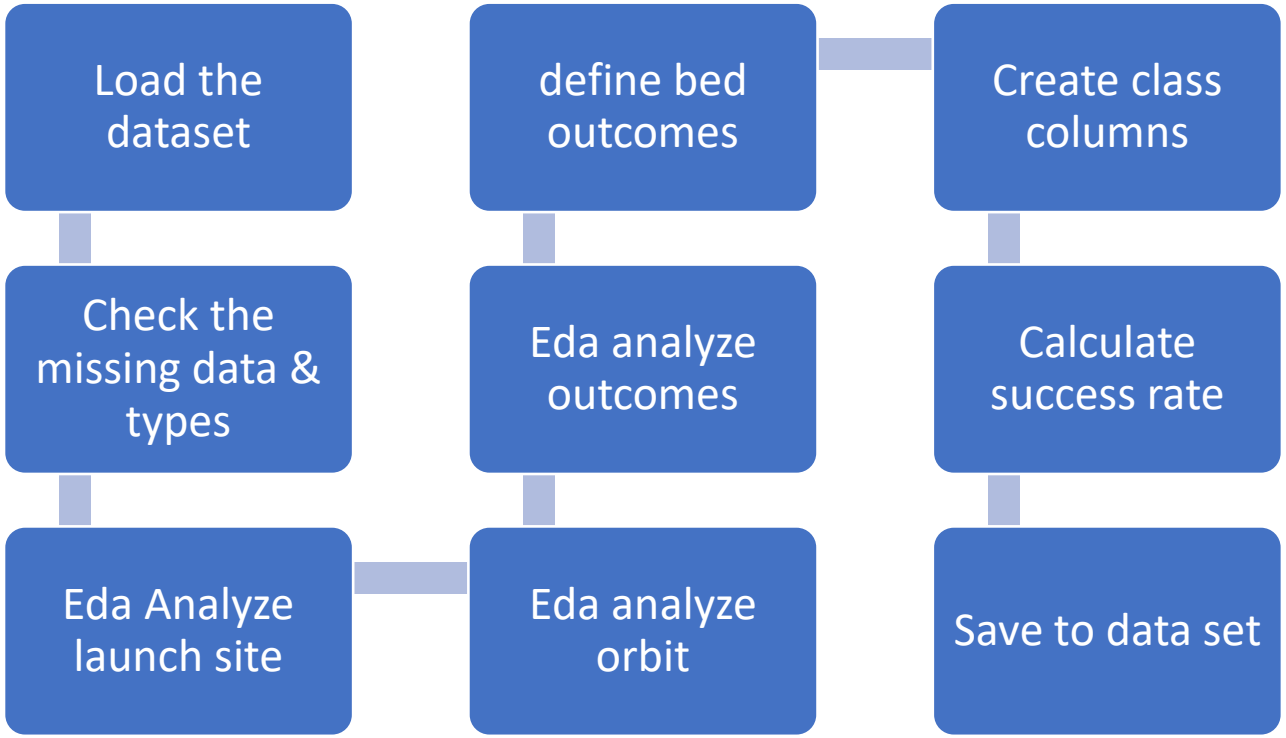


Data Wrangling

- **Describe how data were processed]**

1. **Load Data:** Loaded the dataset_part_1.csv
2. **Perform EDA:** Analyzed the data by checking types (.dtypes) and calculating value counts (.value counts()) for Launch Site, Orbit, and Outcome columns.
3. **Define Success:** Based on the Outcome analysis, we created a set of bad outcomes (like None None, False ASDS, False RTLS, etc.).
4. **Create Target Variable:** We iterated through the Outcome column and created a new binary **Class** column.
5. **Assign Labels:** Assigned a **0** (failure) if the outcome was in bad outcomes, and a **1** (success) otherwise.
6. **Final Output:** Calculated the final success rate (66.7%) and saved the new Data Frame (with the Class column) as **dataset_part_2.csv**.

You need to present your data wrangling process using key phrases and flowcharts



https://github.com/viraj01-coder/data-science-capstone/blob/main/labs-jupyter-spacex-Data%20wrangling-v2%20(1).ipynb

EDA with Data Visualization

- Summarize what charts were plotted and why you used those charts
- **Scatter Plots:** We used scatter plots with Class (success/failure) as the hue (color). This was done to identify patterns between success (1) and failure (0) for features like Payload Mass and Orbit.
- **Bar Chart:** We created a bar chart to compare the **average success rate** for each Orbit type. This clearly showed which orbits had the highest probability of success.
- **Line Chart:** We plotted a line chart of Year vs. **Average Success Rate**. This was essential to show the trend of how SpaceX's landing success improved over time.
- **Feature Engineering:** Finally, we used pd.get_dummies() to convert text columns (like Orbit) into numeric columns (0/1) and saved the result as dataset_part_3.csv for the next lab.
- https://github.com/viraj01-coder/data-science-capstone/blob/main/jupyter-labs-eda-dataviz-v2%20(1).ipynb

EDA with SQL

Using bullet point format, summarize the SQL queries you performed

- **Data Setup:** First, we loaded the Spacex.csv data into a Pandas Data Frame and then loaded it into a SQL table named SPACEXTABLE in a SQLite database.

- **Basic Queries (Filtering):**

1. Used WHERE and LIKE 'CCA%' to filter records for launch sites starting with 'CCA'.
2. Used SELECT DISTINCT to find all unique Launch Site names.

.

Aggregate Functions (Aggregations):

1. Used SUM(PAYLOAD_MASS__KG_) to calculate the total payload mass for 'NASA (CRS)' customers
2. Used AVG(PAYLOAD_MASS__KG_) to find the average payload for a specific Booster Version ('F9 v1.1').
3. Used MIN(Date) to find the earliest date of a successful 'ground pad' landing.
4. Used COUNT(Mission Outcome) with GROUP BY to count the total number of successful vs. failed missions.

Subqueries & Dates:

1. Used a **subquery** (SELECT max(...)) to find the Booster Version that carried the maximum payload.
2. Used the substr(Date, ...) function to filter and display failed drone ship landings that occurred in the year 2015.
3. Used BETWEEN to filter records within a specific date range and ORDER BY ... DESC to rank the Landing Outcome counts.

https://github.com/viraj01-coder/data-science-capstone/blob/main/jupyter-labs-eda-sql-coursera_sqllite.ipynb

Build an Interactive Map with Folium

- Summarize what map objects such as markers, circles, lines, etc. you created and added to a folium map

folium. Map: We created a base map of the world, centered on the launch sites.

folium. Marker & folium. Circle: We added markers and circles to pin the exact geographic coordinates of each SpaceX launch site (e.g., CCAFS, KSC, VAFB).

Marker Cluster: We used a marker cluster object to group the large number of individual launch markers from the same site.

folium. Icon (with custom colors): We created custom icons for the markers. The icon color was set to **'green'** for a successful launch (Class 1) and **'red'** for a failed launch (Class 0) using a helper function.

folium. Polyline: We drew lines (e.g., in red or blue) from a launch site to nearby proximities (like the coastline and a railway).

folium.DivIcon: We created custom markers to display the calculated distance (in KM) to these proximities directly on the map.

Mouse Position: We added a plugin to show live latitude and longitude coordinates on mouse-over.

Explain why you added those objects

Markers & Circles: These were used to visually locate and identify the different launch sites on the map.

Marker Cluster: This was essential to manage the cluttered view, as many launches happen from the same location. It allows a user to "zoom in" to see the individual launch outcomes.

Green/Red Icons: This was the primary goal. The colors allow us to **visually analyze the success/failure rate** of each site. A cluster of green markers instantly shows a high success rate.

PolyLines & DivIcons: These were used to **measure and visualize the distance** from a launch site to key infrastructure (railway) and geographical features (coastline), helping to analyze operational proximities.

https://github.com/viraj01-coder/data-science-capstone/blob/main/lab-jupyter-launch-site-location-v2%20(1)%20(2).ipynb

Build a Dashboard with Plotly Dash

- **Summarize what plots/graphs and interactions you have added to a dashboard**

Interactions (The "Inputs"):

dcc.Dropdown: We added a dropdown menu to allow users to select a specific **Launch Site** or 'ALL' sites.

- **dcc.RangeSlider:** We added a range slider to allow users to filter the data based on a selected **Payload Mass (kg)** range (from 0 to 10,000).

Plots/Graphs (The "Outputs"):

dcc.Graph (Pie Chart): We added a pie chart to visualize the total successful (Class 1) vs. failed (Class 0) launches for the site selected in dropdown

dcc.Graph (Scatter Plot): We added a scatter plot to show the correlation between Payload Mass and Class (success/failure), colored by Booster Version.

Explain why you added those plots and interactions

Why Interactions? The Dropdown and Slider are the main **interactive components**. They allow the user to dynamically filter the dataset.

Why Pie Chart? The pie chart was added to provide an immediate, clear visual summary of the **overall success rate** for the specific site the user selects.

Why Scatter Plot? The scatter plot was added to analyze a deeper relationship: **"Does payload** weight affect landing success for a specific site?“

The "Callback" (@app.callback): We used callbacks to make the dashboard interactive.

1. The Pie Chart **updates** based on the site-dropdown input.
2. The Scatter Plot **updates** based on *both* the site-dropdown and the payload-slider inputs, allowing for a more specific, two-level filtering.

https://github.com/viraj01-coder/data-science-capstone/blob/main/spacex-dash-app.py/

Predictive Analysis (Classification)

- **<u>Summarize how you built, evaluated, improved, and found the best performing classification model</u>**
- **Data Preparation:**

1. Loaded the features (X) from dataset_part_3.csv (One-Hot Encoded data) and the target (Y) from dataset_part_2.csv (the Class column).
2. standardized the features (X) using **StandardScaler**.
3. Split the data into training and testing sets (80% train, 20% test) using **train_test_split**.

Model Building & Tuning:

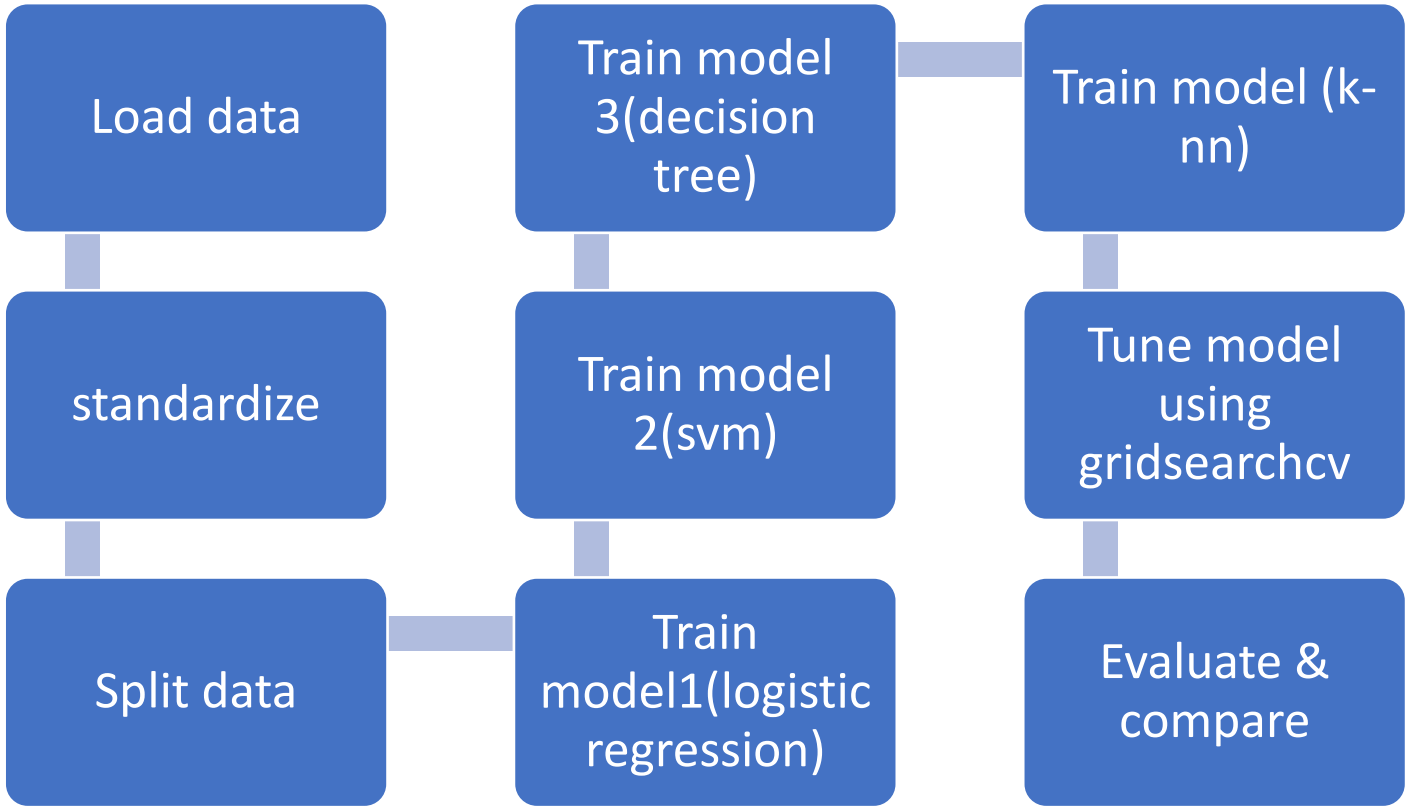
We built, trained, and tuned (using **GridSearchCV** with 10-fold cross-validation) four different classification models:

1. Logistic Regression
2. Support Vector Machine (SVM)
3. Decision Tree Classifier
4. K-Nearest Neighbors (k-NN)

Model Evaluation & Selection:

1. We evaluated each tuned model's accuracy on he unseen the data.
2. **Result:** In our experiment, all four models **performed identically** on the test set, each achieving a final accuracy of **83.33%**.
3. We also plotted the **Confusion Matrix** for each model to analyze the 'True Positive' (landed) and 'False Positive' (did not land) predictions.

You need present your model development process using key phrases and flowchart



https://github.com/viraj01-coder/data-science-capstone/blob/main/SpaceX-Machine-Learning-Prediction-Part-5-v1.ipynb.

Results

- **Exploratory data analysis results**

"Our analysis found that landing success is strongly related to the Orbit type, Launch Site, and PayloadMass. We also observed that the success rate dramatically improved after 2015."

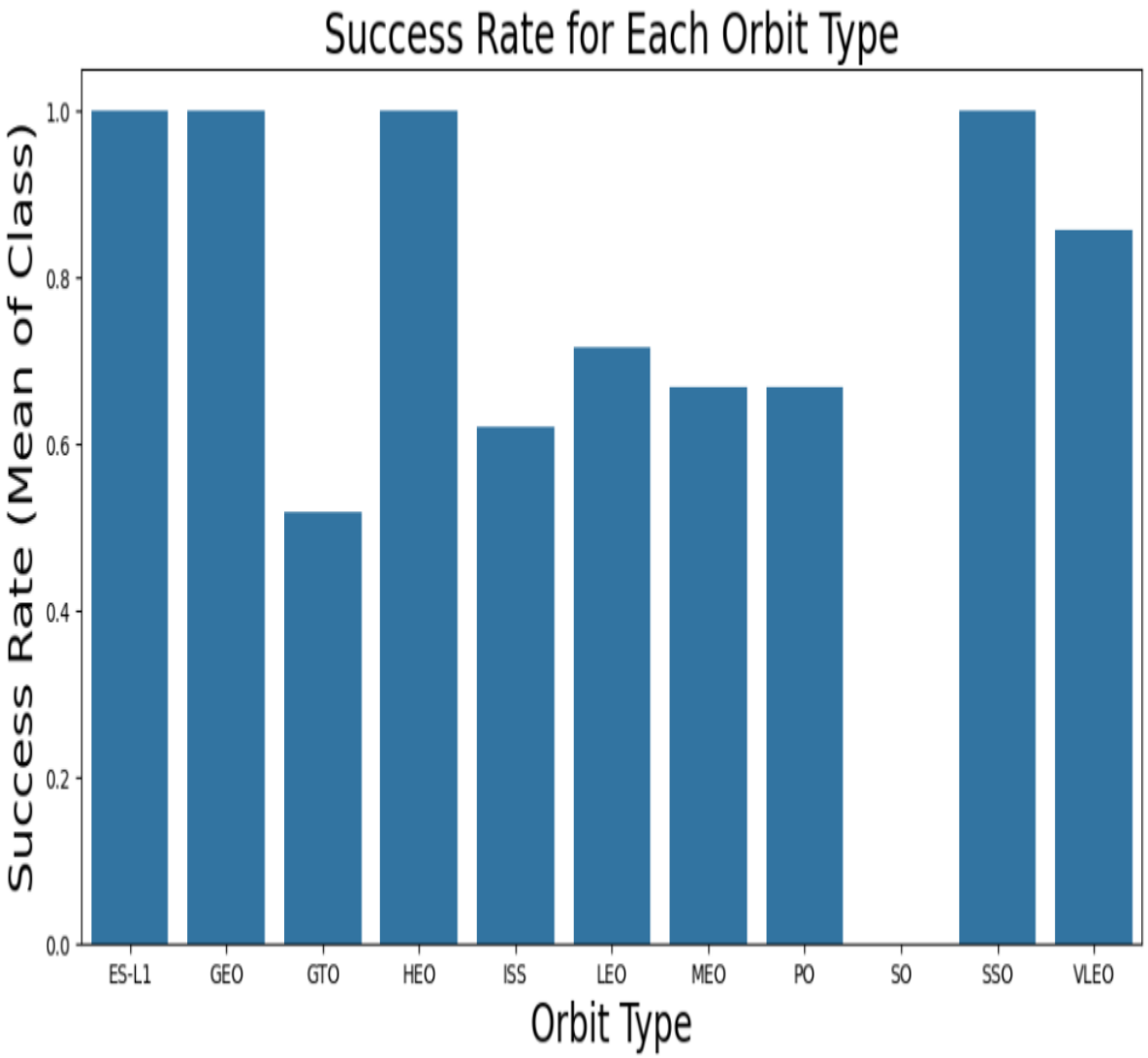
- Interactive analytics demo in screenshots

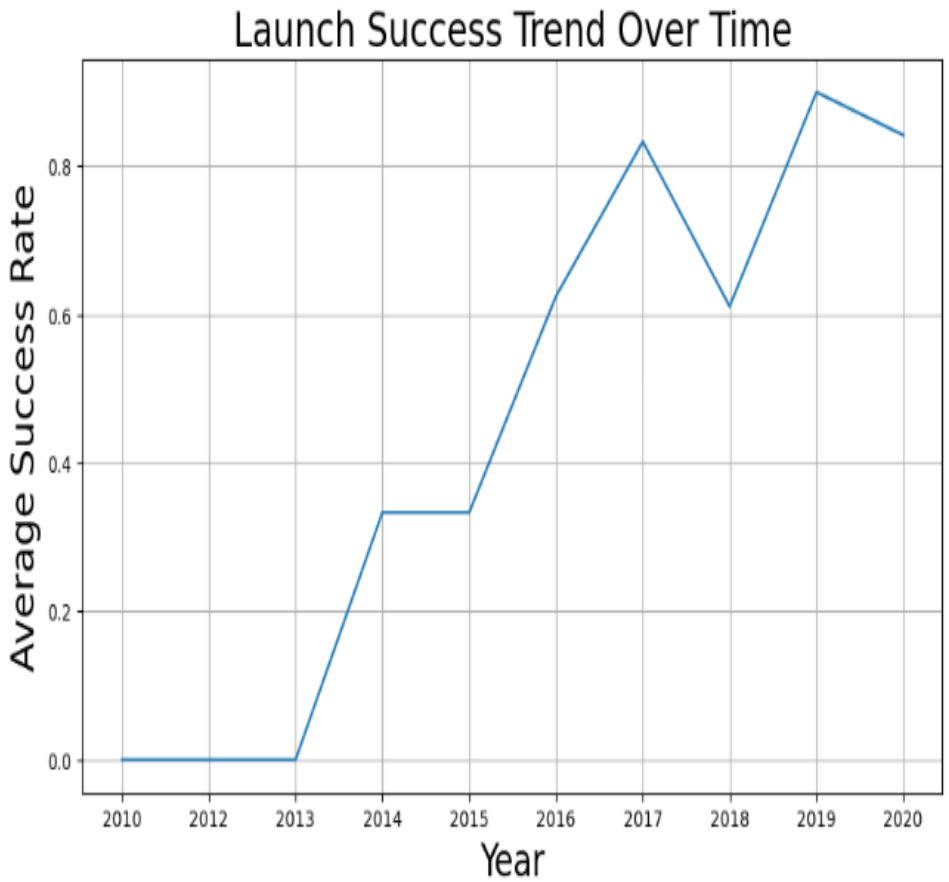
1. A **Folium Map** to visualize launch site success rates with 'green' (success) and 'red' (failure) markers.
2. A **Plotly Dash Dashboard** to filter launch success pie charts and scatter plots by LaunchSite and PayloadMass."

- Predictive analysis results

"We trained and tuned 4 classification models (Logistic Regression, SVM, Decision Tree, k-NN). All four models achieved an identical test accuracy of **83.33%** in predicting a successful landing.

Exploratory data analysis results

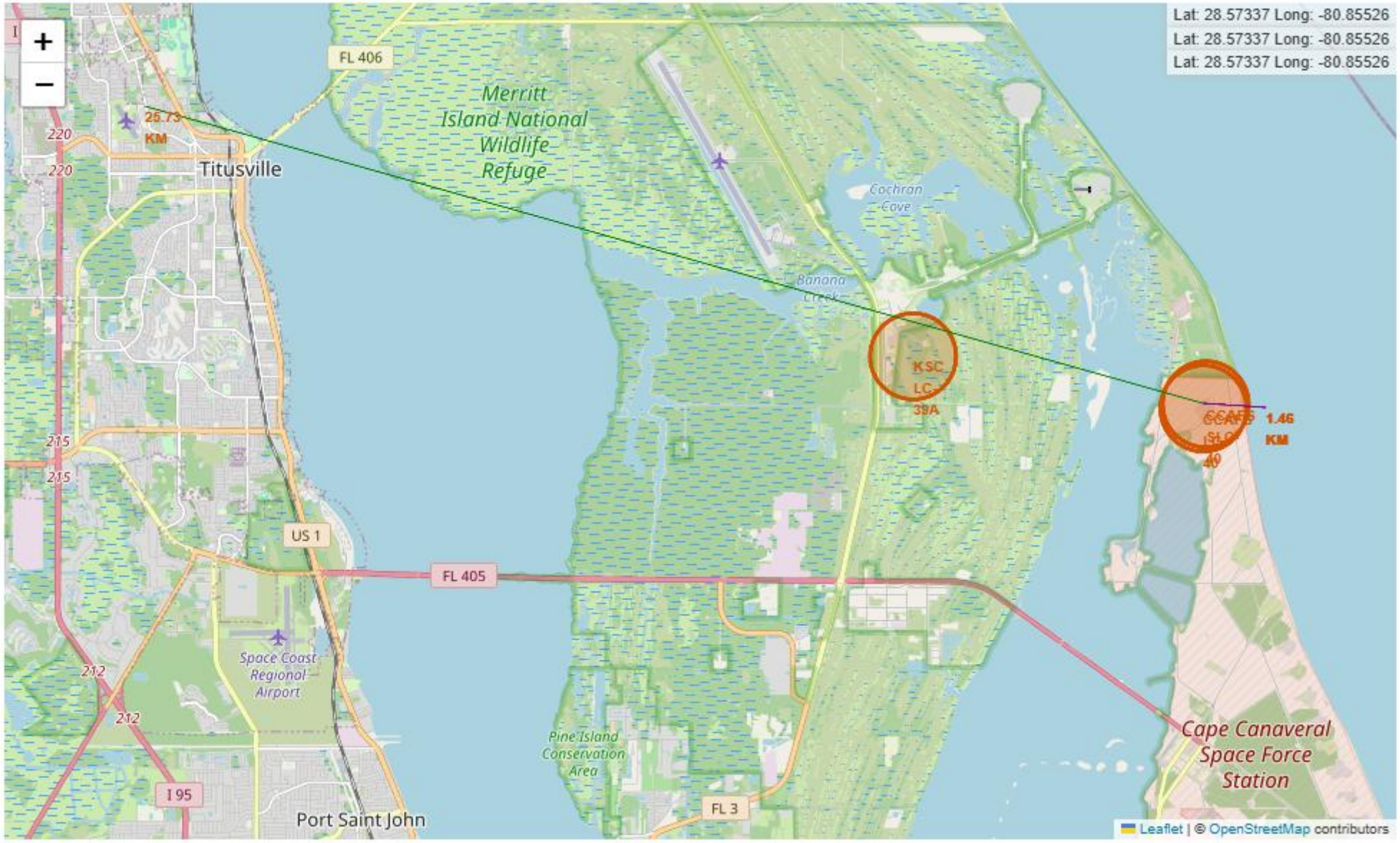


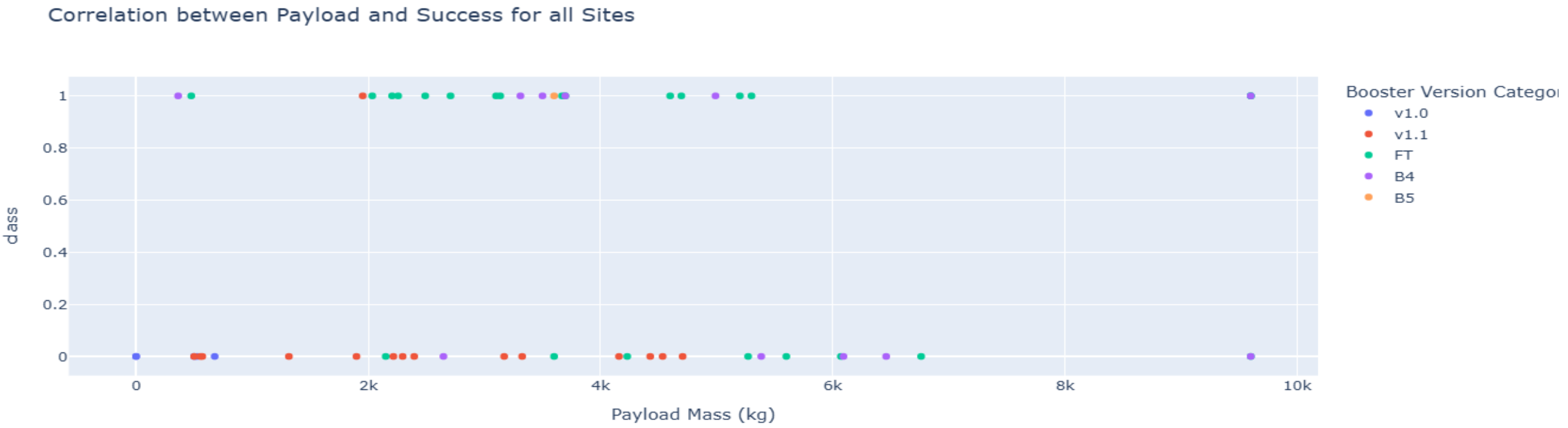


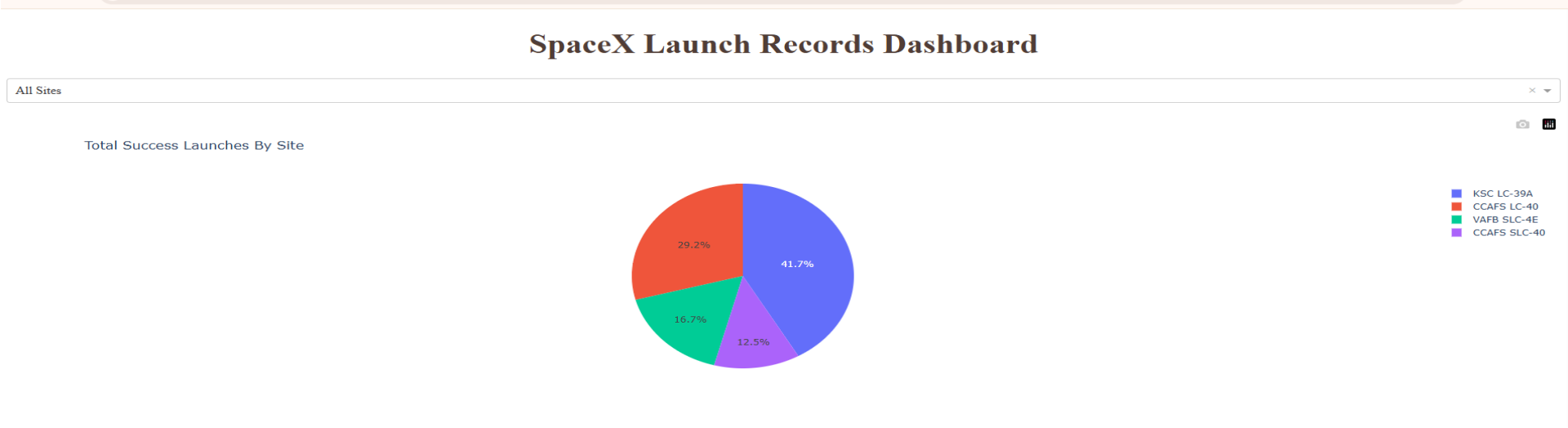
You can observe that the success rate since 2013 kept increasing till 2017 (stable in 2014) and after 2015 it started increasing.

Interactive analytics demo in screenshots

[76]:




Correlation between Payload and Success for all Sites
class
1
0.8
0.6
0.4
0.2
0
0
2k
4k
6k
8k
10k
Payload Mass (kg)
Booster Version Categor
v1.0
v1.1
FT
B4
B5


SpaceX Launch Records Dashboard
All Sites
Total Success Launches By Site
29.2%
41.7%
16.7%
12.5%
KSC LC-39A
CCAFS LC-40
VAFB SLC-4E
CCAFS SLC-40

Predictive analysis results

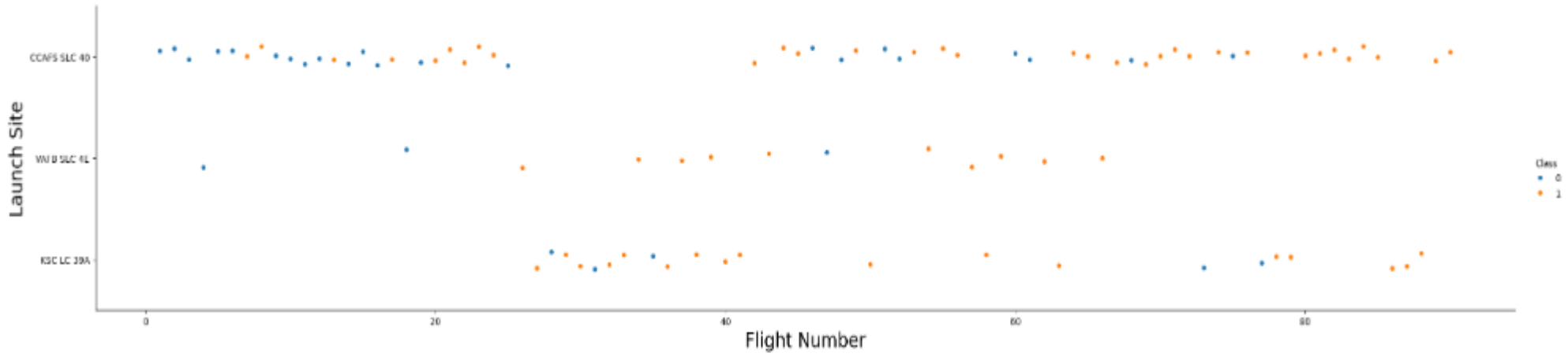
```
Logistic Regression Accuracy: 0.8333333333333334
SVM Accuracy: 0.8333333333333334
Decision Tree Accuracy: 0.8333333333333334
K-Nearest Neighbors Accuracy: 0.8333333333333334
```

Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

- Show a scatter plot of Flight Number vs. Launch Site



Now try to explain the patterns you found in the Flight Number vs. Launch Site scatter point plots.

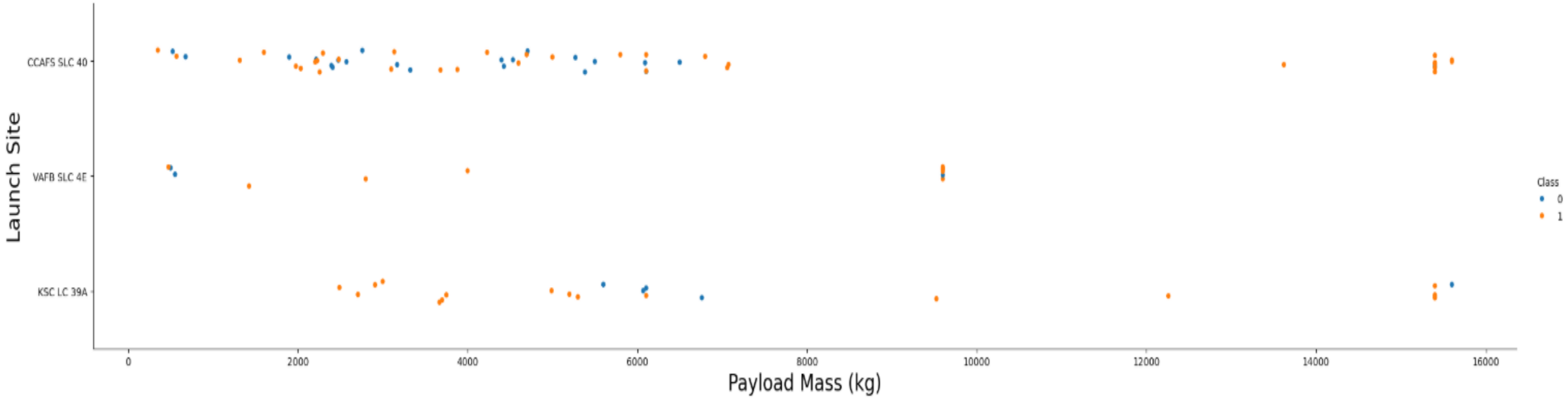
The key insight is that as the `Flight Number` increases (moving right), the proportion of successful (orange) dots increases, showing that SpaceX's success rate improved over time.

This scatter plot maps Flight Number against LaunchSite, using color (hue) to represent the Class (0=Blue/Fail, 1=Orange/Success).

The key insight is that as the `Flight Number` increases (moving right), the proportion of successful (orange) dots increases, showing that SpaceX's success rate improved over time.

Payload vs. Launch Site

- Show a scatter plot of Payload vs. Launch Site

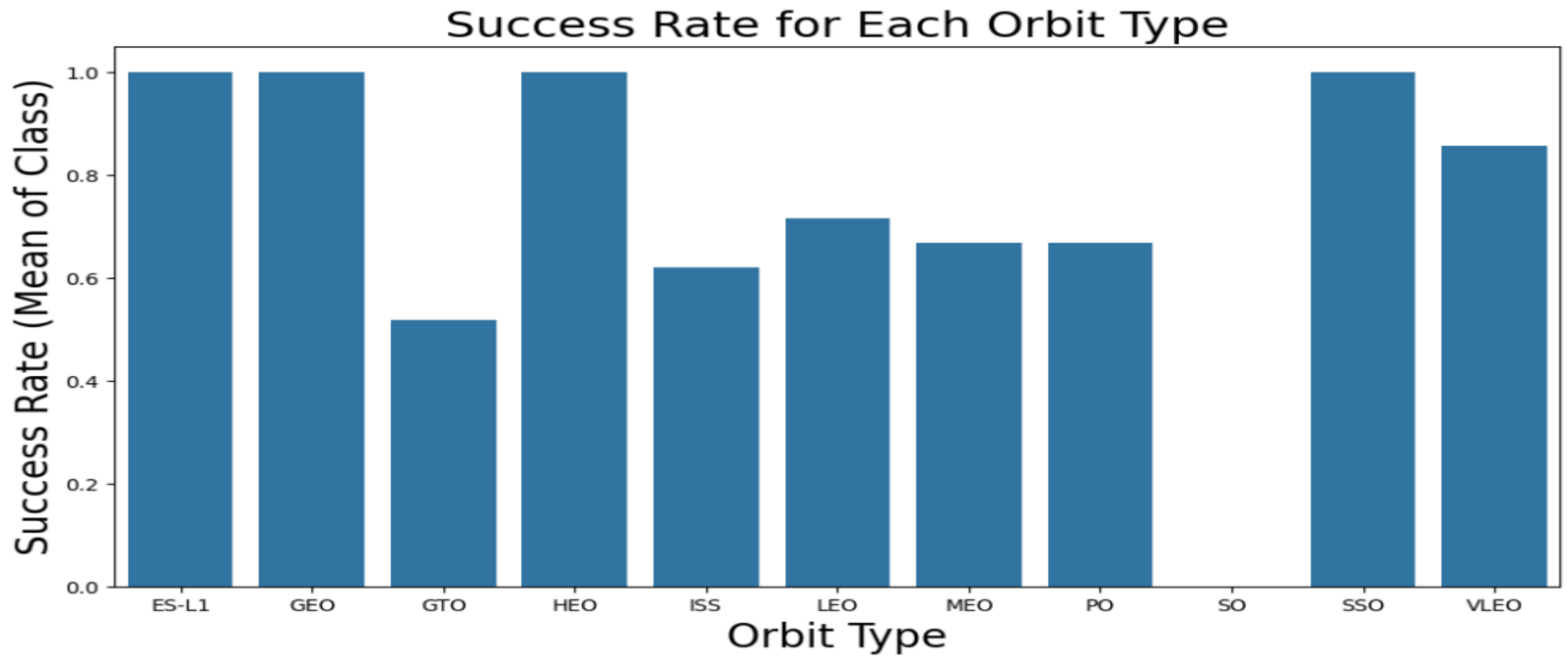


Now if you observe Payload Vs. Launch Site scatter point chart you will find for the VAFB-SLC launchsite there are no rockets launched for heavypayload mass(greater than 10000).

What it Shows: This plot maps Payload Mass (kg) against Launch Site, using color (hue) to show Class (0=Blue/Fail, 1=Orange/Success).
Key Insight: It reveals that the **KSC LC 39A** site has a very high success rate across various payload masses, while other sites show different patterns.
Conclusion: This shows that Launch Site and Payload Mass are important factors for predicting landing success.

Success Rate vs. Orbit Type

- Show a bar chart for the success rate of each orbit type

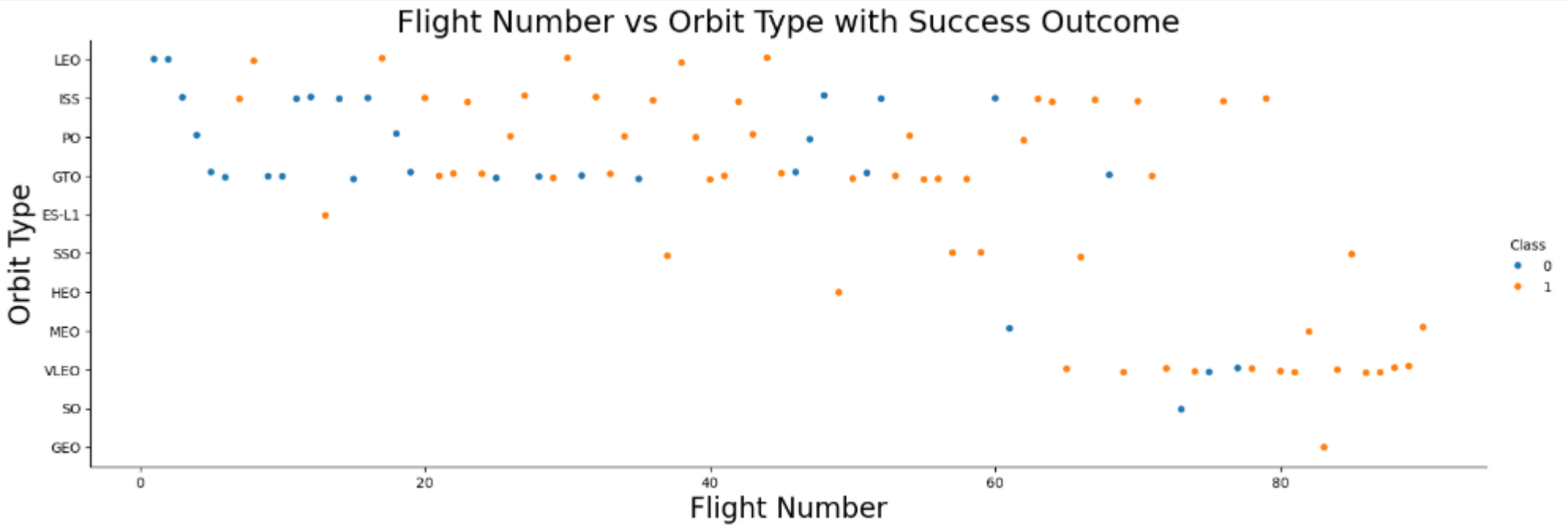


Analyze the ploted bar chart try to find which orbits have high sucess rate.

What it Shows: This bar chart compares the **average success rate** (the mean of the Class column) for each different Orbit Type.
Key Insight: It clearly shows that Orbit is a major factor. Orbits like **ES-L1**, **GEO**, **HEO**, and **SSO** have a near 100% success rate.
Conclusion: In contrast, the **GTO** orbit has the lowest success rate (around 52%), making the Orbit Type a very strong predictor of landing success.

Flight Number vs. Orbit Type

- Show a scatter point of Flight number vs. Orbit type

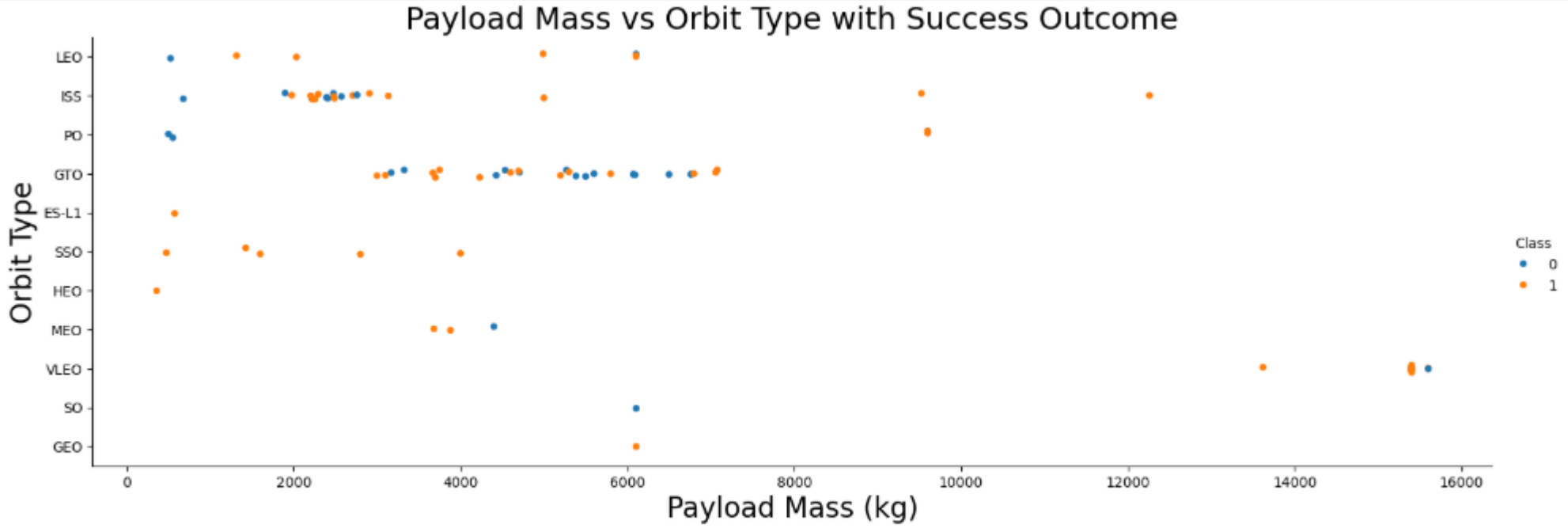


You should see that in the LEO orbit the Success appears related to the number of flights; on the other hand, there seems to be no relationship between flight number when in GTO orbit.

What it Shows: This scatter plot maps Flight Number (X-axis) against Orbit Type (Y-axis), using color (hue) to show Class (0=Blue/Fail, 1=Orange/Success).
Key Insight: The plot shows that the success rate evolved differently for each orbit. For example, in the **LEO** orbit, success (orange dots) becomes much more common as the Flight Number increases.
Conclusion: This suggests that SpaceX's success rate improved over time, but this trend is more obvious in some orbits (like LEO) than in others.

Payload vs. Orbit Type

- Show a scatter point of payload vs. orbit type



With heavy payloads the successful landing or positive landing rate are more for Polar,LEO and ISS.

However for GTO we cannot distinguish this well as both positive landing rate and negative landing(unsuccessful mission) are both there here.

GTO Orbit: This orbit shows a mix of both successes (orange) and failures (blue) across a wide range of payload masses. There is no clear pattern, as both successful and unsuccessful missions are present.
LEO & ISS Orbits: In contrast, the LEO and ISS orbits show a very high success rate (mostly orange dots) across their respective payload ranges.
SSO Orbit: The SSO orbit appears to be used primarily for heavier payloads, and these launches have also been highly successful.
Conclusion: This chart demonstrates that the Orbit Type and Payload Mass interact in complex ways, and both are critical factors for predicting a successful landing.

Launch Success Yearly Trend

- Show a line chart of yearly average success rate

You can observe that the success rate since 2013 kept increasing till 2017 (stable in 2014) and after 2015 it started increasing.

What the Plot Shows: This line chart tracks the **Average Success Rate** (Y-axis) of SpaceX's landings for each Year (X-axis) from 2010 to 2020.
2010-2013: The success rate was 0%, reflecting the early failures.
2014-2017: After 2014, the rate shows a steep and consistent increase, peaking at over 80% in 2017.
2018-2020: Despite a small dip in 2018, the overall trend is clearly **positive** and upward.
Conclusion: This is one of the most important findings. It clearly visualizes how SpaceX's landing technology matured and became significantly more reliable over the decade.

All Launch Site Names

- Find the names of the unique launch sites

Display the names of the unique launch sites in the space mission

```
[20]: %sql select distinct "Launch_Site" from SPACEXTABLE;
```

```
 * sqlite:///my_data1.db
Done.
```

[20]:

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

We used the `SELECT DISTINCT` command to query the `SPACEXTABLE`. This command scans the entire `Launch Site` column and returns only the unique values, showing us all the different launch sites used in the dataset."

Launch Site Names Begin with 'CCA'

- Find 5 records where launch sites begin with `CCA`

Display 5 records where launch sites begin with the string 'CCA'

[22]:
```
%sql select * from SPACEXTABLE where "Launch_Site" like 'CCA%' limit 5;
```

```
 * sqlite:///my_data1.db
Done.
```

[22]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

What the Query Does: We used the SELECT * command to get all columns from the table
The 'WHERE' Clause: We used the WHERE clause combined with the like operator to perform pattern matching
The 'LIKE' operator: The pattern LIKE 'CCA%' searches the Launch Site column and finds any text that **starts with "CCA"**. The % is a wildcard character that means "anything can come after".
The 'LIMIT 5': We used LIMIT 5 to restrict the output to only the first 5 records that matched the "CCA" pattern.

Total Payload Mass

- Calculate the total payload carried by boosters from NASA

Display the total payload mass carried by boosters launched by NASA (CRS)

```
[24]: %sql select sum(PAYLOAD_MASS__KG_) from SPACEXTABLE where Customer = 'NASA (CRS)';
```

```
 * sqlite:///my_data1.db
Done.
```

[24]:

sum(PAYLOAD_MASS__KG_)
45596

What it does: This SQL query uses the SUM() aggregate function.
How: It adds up all the values in the PAYLOAD_MASS__KG_ column.
Filter: The WHERE Customer = 'NASA (CRS)' clause ensures it *only* sums the values for rows where the customer is **'NASA (CRS)'**.
Result: The total payload mass launched for NASA (CRS) is **45596 kg**.

Average Payload Mass by F9 v1.1

- Calculate the average payload mass carried by booster version F9 v1.1

Display average payload mass carried by booster version F9 v1.1

```
[26]: %sql select avg(PAYLOAD_MASS__KG_) from SPACEXTABLE where "Booster_Version" = 'F9 v1.1';

 * sqlite:///my_data1.db
Done.
```

[26]:

avg(PAYLOAD_MASS__KG_)
2928.4

What it does: This SQL query uses the AVG() aggregate function.
How: It calculates the **average** of all values in the PAYLOAD_MASS__KG_ column.
Filter: The WHERE "Booster Version" = 'F9 v1.1' clause ensures it *only* averages the values for rows where the booster was **'F9 v1.1'**.
Result: The average payload mass carried by the 'F9 v1.1' booster is **2928.4 kg**.

First Successful Ground Landing Date

- Find the dates of the first successful landing outcome on ground pad

List the date when the first succesful landing outcome in ground pad was acheived.

Hint:Use min function

```
[28]: %sql select min(Date) from SPACEXTABLE where "Landing_Outcome" = 'Success (ground pad)';
```

```
 * sqlite:///my_data1.db
Done.
```

[28]:

min(Date)
2015-12-22

What it does: This SQL query uses the MIN() aggregate function.
How: It finds the **earliest** (minimum) value in the Date column.
Filter: The WHERE clause restricts this search *only* to rows where the Landing Outcome was **'Success (ground pad)'**.
Result: The date of the very first successful ground pad landing was **2015-12-22**.

Successful Drone Ship Landing with Payload between 4000 and 6000

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
[30]: %sql select "Booster_Version" from SPACEXTABLE where "Landing_Outcome" = 'Success (drone ship)' and PAYLOAD_MASS__KG_ between 4000 and 600
```

```
 * sqlite:///my_data1.db
Done.
```

[30]:

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

What it does: This query selects the names of boosters (Booster_Version).
Filter 1 (Landing): The WHERE clause first filters the results to include *only* rows where the Landing_Outcome was **'Success (drone ship)'**.
Filter 2 (Payload): The AND operator adds a second filter: the PAYLOAD_MASS__KG_ must also be **BETWEEN 4000 and 6000**.
Result: The list shows the four boosters that successfully landed on a drone ship *and* were carrying a payload in that specific 4000-6000 kg range.

Total Number of Successful and Failure Mission Outcomes

- Calculate the total number of successful and failure mission outcome

```
[32]: %sql select "Mission_Outcome", count("Mission_Outcome") as Total from SPACEXTABLE group by "Mission_Outcome";
```

```
 * sqlite:///my_data1.db
Done.
```

[32]:

Mission_Outcome	Total
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

This SQL query uses the **GROUP BY** command to bundle all identical Mission Outcome entries together.
It then uses **COUNT()** to count how many rows are in each unique group (e.g., all "Success" rows, all "Failure" rows).

Boosters Carried Maximum Payload

- List the names of the booster which have carried the maximum payload mass

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

[34]:
```
%sql select "Booster_Version" from SPACEXTABLE where PAYLOAD_MASS__KG_ = (select max(PAYLOAD_MASS__KG_) from SPACEXTABLE);
```

```
 * sqlite:///my_data1.db
Done.
```

[34]:

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

What it does: This query uses a **subquery** to find the answer in two steps.
Inner Query (Subquery): The (select max(PAYLOAD_MASS__KG_) from SPACEXTABLE) part runs first. It finds the single *highest payload mass* value in the entire table (e.g., 15,600 kg).
Outer Query: The main query then selects all Booster Version names where the PAYLOAD_MASS__KG_ is **equal to** the maximum value found by the inner query.
Result: The list shows all the specific boosters that have carried that top payload weight.

2015 Launch Records

- List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
%sql select substr(Date, 6, 2) as Month, "Landing_Outcome", "Booster_Version", "Launch_Site" from SPACEXTABLE where "Landing_Outcome" = 'F
```

```
 * sqlite:///my_data1.db
Done.
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

What it does: This query selects the Month, Landing Outcome , Booster Version, and Launch Site.
How: It uses the substr() function (as required by SQLite) to extract parts of the Date string for filtering.
Filter 1: The WHERE clause first finds all records where the Landing Outcome was **'Failure (drone ship)'**.
Filter 2: The AND clause adds a second filter: substr(Date, 0, 5) = '2015' finds all records where the **year** was **2015**.
Result: The list shows the two drone ship failures that happened in 2015, occurring in Month 01 (January) and 04 (April).

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

[38]:
```
%sql select "Landing_Outcome", count("Landing_Outcome") as Count_Outcome from SPACEXTABLE where Date between '2010-06-04' and '2017-03-20'
```

```
 * sqlite:///my_data1.db
Done.
```

[38]:

Landing_Outcome	Count_Outcome
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

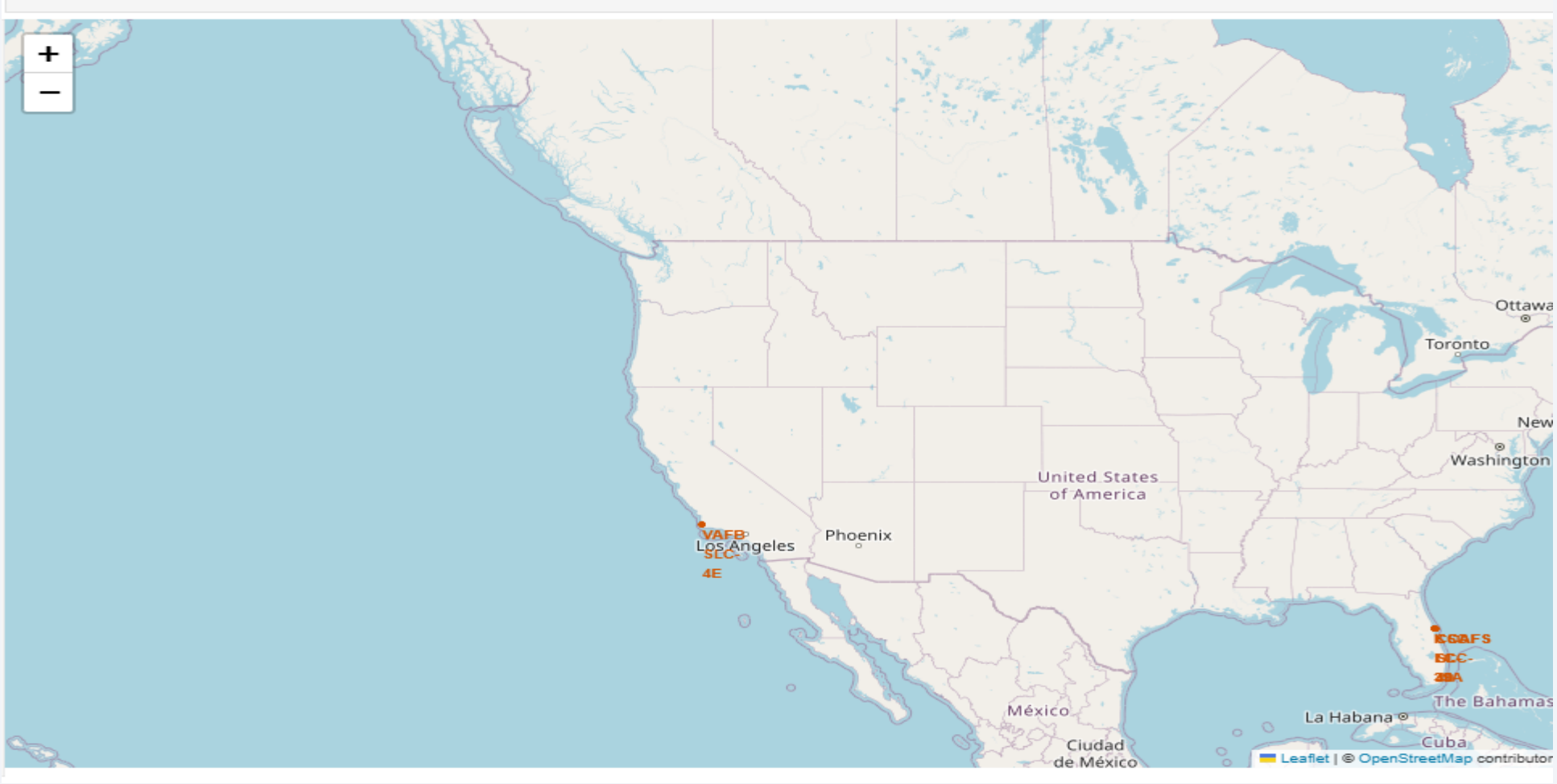
What it does: This query counts and ranks the different types of `Landing Outcome`.
Filter (Date): The WHERE Date BETWEEN ... clause filters the data to include *only* the records from 2010 to March 2017.
Grouping & Counting: It uses GROUP BY "Landing Outcome" to group similar outcomes and COUNT() to get the total for each group.
Ordering: The query (as implied by the task description) uses ORDER BY ... DESC to rank the outcomes, showing the most common ones ('No attempt’) at the top.
Result: The list shows the total count for each landing outcome that occurred *within* that specific 2010-2017 period.

Section 3

Launch Sites Proximities Analysis

<All Launch Site Locations on Map>

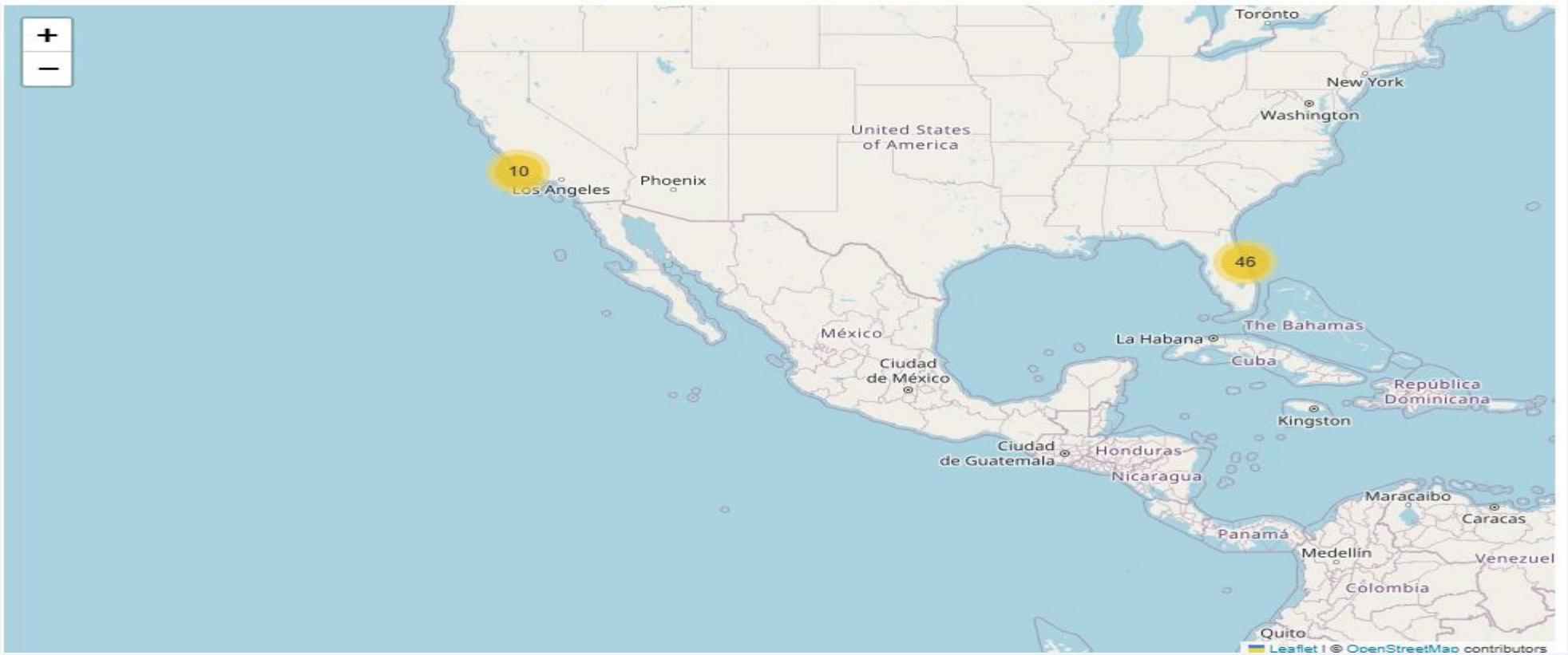
- Explore the generated folium map and make a proper screenshot to include all launch sites' location markers on a global map



Elements: We first created a `folium. Map` centered on the USA. We then looped through our `launch_sites_df` and added a `folium. Marker` and `folium. Circle` for each of the 3 unique SpaceX launch sites.
Findings: This map visually confirms the geographic locations of all launch sites: CCAFS and KSC are clustered together on the coast of Florida, while VAFB is located on the coast of California.

<Launch Success/Failure Markers>

- Explore the folium map and make a proper screenshot to show the color-labeled launch outcomes on the map



Elements: We created a Marker Cluster to handle the large number of launches from the same location.
Success/Failure: We created a helper function (assign_marker_color) to set the marker color based on the Class column:
Class 1 (Success)
Class 0 (Failure)
Findings: The map is now interactive. The **yellow clusters** show the total number of launches from each site (e.g., 46 from Florida, 10 from California). When we zoom in, we can see the individual **green (success)** and **red (failure)** markers, allowing us to visually analyze the success rate of each site.

<Folium Map Screenshot 3>

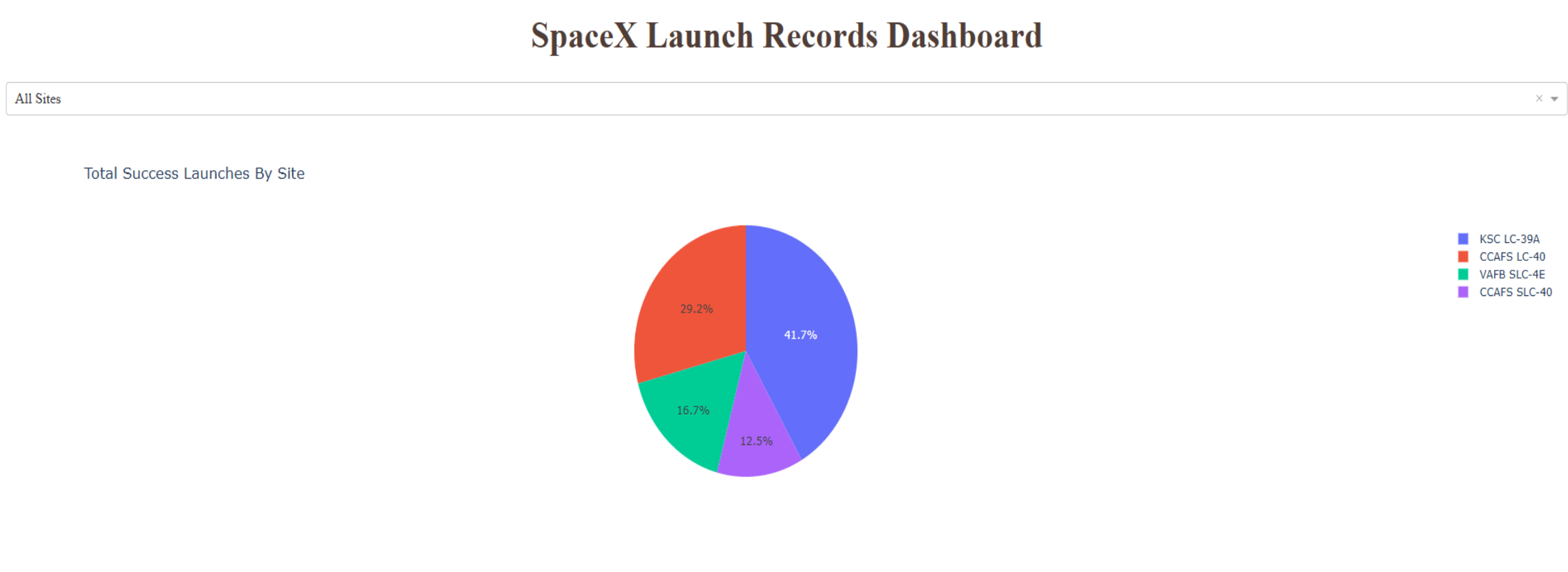
- Replace <Folium map screenshot 3> title with an appropriate title

Elements : We used the Mouse Position tool to find the coordinates for the closest **Railway**, **Highway** (Samuel C Phillips Pkwy), and **City** (Titusville)
We then added folium. Polyline (the colored lines) and folium.DivIcon (the distance in KM) to the map to visualize these proximities.
Findings : This map visually confirms that the CCAFS launch site is strategically placed. It is very close (approx. 1.46 KM) to essential infrastructure like the highway, while maintaining a safe buffer distance (approx. 20.73 KM) from the nearest city (Titusville).

Section 4

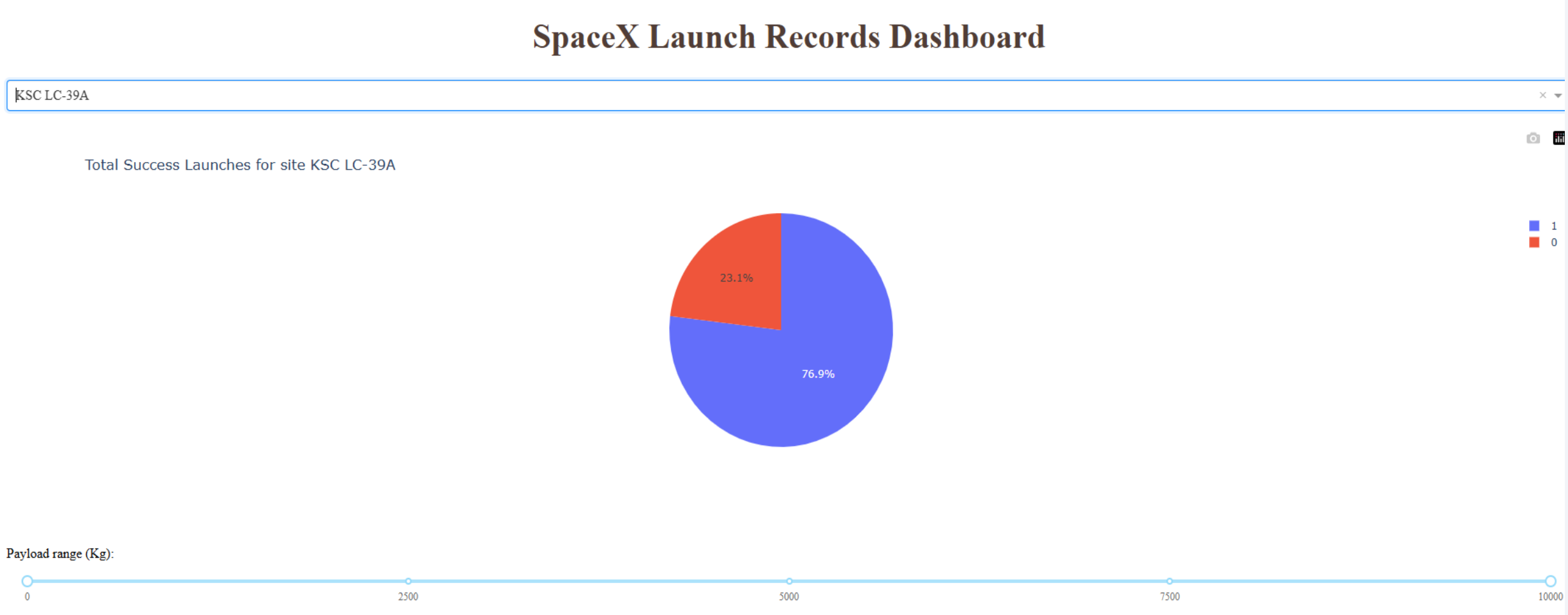
Build a Dashboard with Plotly Dash

<Dashboard: Total Success Launches by Site (Pie Chart)>



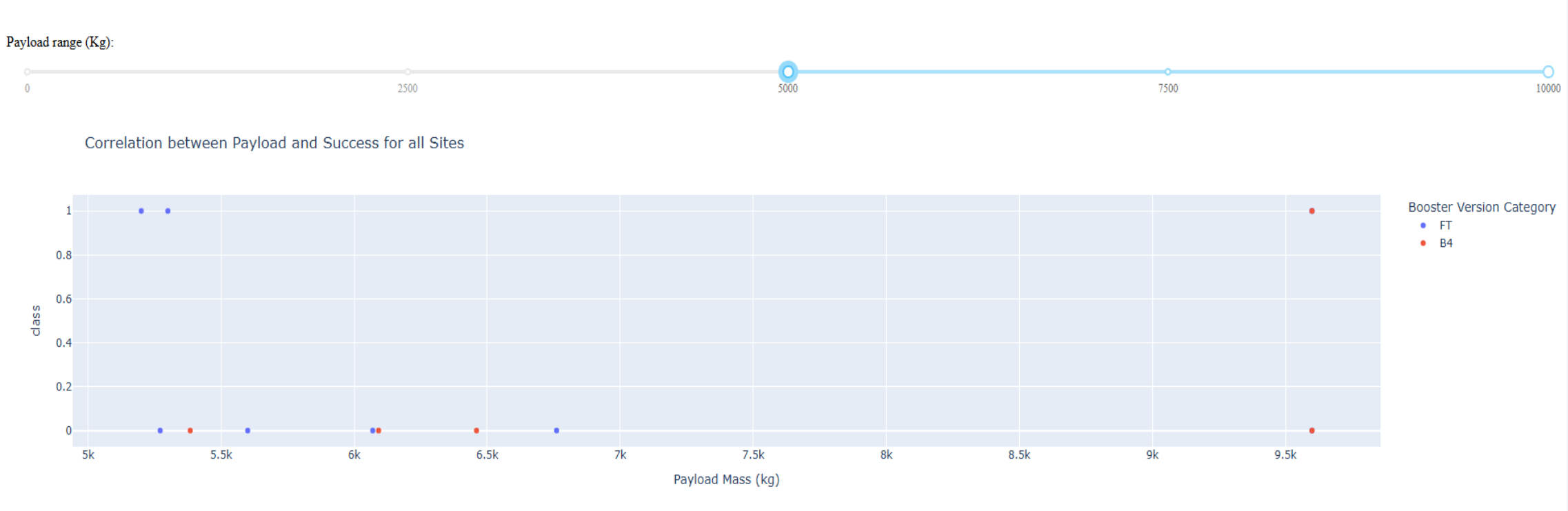
Elements: This screenshot shows the first component of our dashboard: a **Pie Chart** (a dcc.Graph) that is being filtered by the dcc.Dropdown (which is set to 'All Sites’).
Findings: The pie chart breaks down all successful launches (Class=1) by their LaunchSite. We can instantly see that **KSC LC-39A** (the blue slice) accounts for the largest portion (**41.7%**) of all successful launches, indicating it is the most historically successful site in the dataset.

<Dashboard: Success Ratio for KSC LC-39A>



Elements: This screenshot shows the Pie Chart after the dcc.Dropdown (Input) was filtered to select only the **'KSC LC-39A'** launch site.
Findings: The Pie Chart has been updated by the callback to show the *specific* success ratio for this site: **76.9% Success** (Class 1, blue) vs. **23.1% Failure** (Class 0, red).
Conclusion: This visually confirms that 'KSC LC-39A' is a high-success launch site, as the 'Success' slice (76.9%) clearly dominates the chart.

<Dashboard: Payload vs. Success>



Elements: This screenshot shows the Scatter Plot (`dcc.Graph`) which is controlled by two inputs: the `dcc.Dropdown` (set to 'ALL Sites') and the `dcc.RangeSlider` (set to a specific range, e.g., 5000-10000 kg).
Interaction: The plot is updated by a callback that filters the Data Frame based on the Payload Mass selected in the slider.
Findings: The color of the dots represents the Booster Version. By filtering for a **heavy payload range (e.g., >5000 kg)**, we can see that the scatter plot updates to show a very **high success rate** (mostly Class 1 dots, colored orange). This indicates that heavy payloads (often on newer 'FT' booster versions) are highly successful across all sites.

Section 5

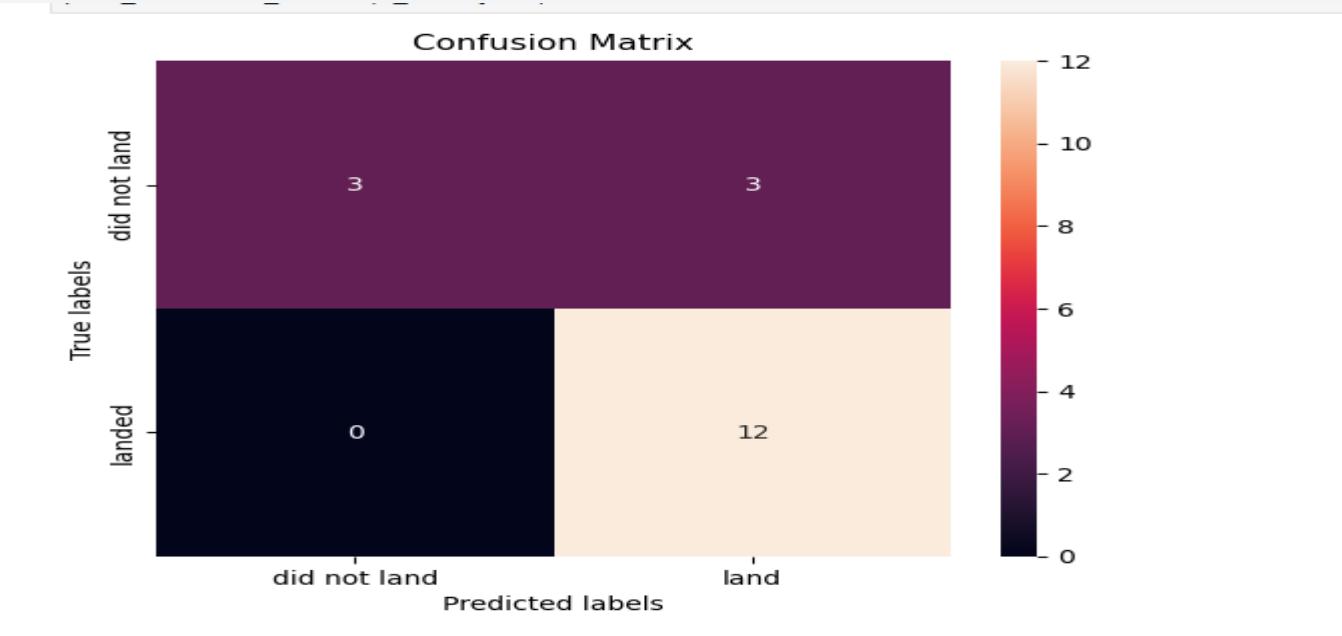
Predictive Analysis (Classification)

Classification Accuracy

Logistic Regression	83.33%
Support Vector Machine (SVM)	83.33%
Decision Tree	83.33%
K-Nearest Neighbors (k-NN)	83.33%

All four classification models (Logistic Regression, SVM, Decision Tree, and k-NN) performed **identically** on the unseen test data after hyperparameter tuning. They all achieved the **same highest accuracy of 83.33%**. Therefore, we cannot designate a single model as the best based solely on this metric, but the Logistic Regression or SVM would typically be preferred due to their simplicity.

Confusion Matrix



The confusion matrix helps us understand the model's performance beyond simple accuracy.

- **True Positives (TP) - [Bottom-Right]: 12**
  - The model correctly predicted **'Landed'** 12 times when the rocket *actually landed*.
- **True Negatives (TN) - [Top-Left]: 3**
  - The model correctly predicted **'Did not land'** 3 times when the rocket *did not land*.
- **False Positives (FP) - [Top-Right]: 3**
  - This is a Type I error. The model incorrectly predicted **'Landed'** 3 times when the rocket *actually did not land*.
- **False Negatives (FN) - [Bottom-Left]: 0**
  - This is a Type II error, and our model has zero. The model **never** incorrectly predicted 'Did not land' when the rocket *actually landed*.

Finding: The model is perfect at identifying all the successful landings (0 False Negatives). Its only mistakes were 3 'False Positives', where it predicted a landing that didn't happen.

Conclusions

- Point 1

SpaceX's landing success rate dramatically improved over time, increasing from 0% in the early years (2010-2013) to an overall success rate of **66.7%** in the dataset, showing clear technological maturation.

- Point 2

The Orbit Type is the strongest predictor of landing success. Orbits like GTO have the lowest success rates, while GEO, HEO, and SSO are nearly 100% successful

- Point 3

- The **LaunchSite** is also a critical factor. The **KSC LC 39A** site shows the **highest success ratio** (around 76.9% success rate) across various payload masses.

- Point 4

- Four different classification models (LR, SVM, Decision Tree, k-NN) were trained and tuned, and all achieved an **identical high test accuracy of 83.33%** in predicting a successful landing outcome.

Appendix

- Include any relevant assets like Python code snippets, SQL queries, charts, Notebook outputs, or data sets that you may have created during this project

TASK 7: Create dummy variables to categorical columns

Use the function `get_dummies` and `features` dataframe to apply OneHotEncoder to the column `Orbits`, `LaunchSite`, `LandingPad`, and `Serial`. Assign the value to the variable `features_one_hot`, display the results using the method head. Your result dataframe must include all features including the encoded ones.

[52]:
```
# HINT: Use get_dummies() function on the categorical columns
categorical_features = ['Orbit', 'LaunchSite', 'Serial']
features_one_hot = pd.get_dummies(features, columns=categorical_features)
features_one_hot.head()
```

[52]:

	FlightNumber	PayloadMass	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Orbit_ES-L1	...	Serial_B1048	Serial_B1049	Serial_B1(
0	1	6104.959412	1	False	False	False	NaN	1.0	0	False	...	False	False	Fa
1	2	525.000000	1	False	False	False	NaN	1.0	0	False	...	False	False	Fa
2	3	677.000000	1	False	False	False	NaN	1.0	0	False	...	False	False	Fa
3	4	500.000000	1	False	False	False	NaN	1.0	0	False	...	False	False	Fa
4	5	3170.000000	1	False	False	False	NaN	1.0	0	False	...	False	False	Fa

5 rows × 76 columns

Thank you!